Aerodynamics Coursework: Potential Flow, Conformal Mapping, Thin Aerofoil Theory and the 2D Panel Method

SID digits used: $N_7 = 3$, $N_8 = 7$, $N_9 = 6$

Task 1: Elementary Potential Flow Solutions

(a) Stream function

The flow is the superposition of a uniform stream U_∞, a source of strength σ at $(-x_0, 0)$ and a sink of strength $-\sigma$ at $(x_0, 0)$. The stream function of a uniform stream is $\psi_\infty = U_\infty y$; the stream function of a 2D source of strength σ at (x_s, y_s) is $\psi = \frac{\sigma}{2\pi}\theta$, with $\theta = \arctan\left(\frac{y-y_s}{x-x_s}\right)$ measured from the source. Superposing all three elementary solutions,

$$\boxed{\psi(x,y) = U_\infty y \; + \; \frac{\sigma}{2\pi}\Big[\operatorname{atan2}(y,\, x+x_0) - \operatorname{atan2}(y,\, x-x_0)\Big]} \tag{1}$$

(b) Streamlines and Rankine body, $U_\infty = 1,\ \sigma = 1,\ x_0 = 1$

The stagnation points lie on the x-axis. Setting $u = \partial\psi/\partial y = 0$ on $y = 0$ gives (using the complex potential $\Phi = U_\infty z + \frac{\sigma}{2\pi}\ln(z + x_0) - \frac{\sigma}{2\pi}\ln(z - x_0)$ and $d\Phi/dz = 0$)

$$x = \pm a, \qquad a = \sqrt{x_0^2 + \frac{\sigma x_0}{\pi U_\infty}} \;=\; 1.1482. \tag{2}$$

The body half-thickness h (at $x = 0$) is found from the stagnation streamline condition $\psi(0, h) = 0$:

$$U_\infty h + \frac{\sigma}{\pi}\arctan\left(\frac{h}{x_0}\right) - \frac{\sigma}{2} = 0 \implies h = 0.3834 \text{ (solved numerically)}. \tag{3}$$

Figure 1 shows the streamlines, the $\psi = 0$ stagnation streamline (which is simultaneously the Rankine-body contour and the dividing streamline along the axis outside the body), and the two stagnation points.

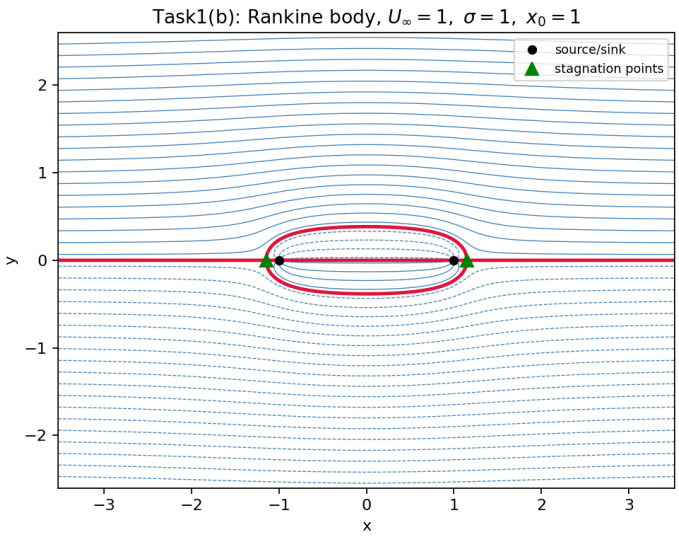


Figure 1: Rankine body for $U_\infty = 1,\ \sigma = 1,\ x_0 = 1$. $a = 1.148$, $h = 0.383$.

(c) Submarine sizing, $L = 50\,\mathrm{m}$, thickness $= 10\,\mathrm{m}$, $U_\infty = 5\,\mathrm{m/s}$

The half-length $a = L/2 = 25\,\mathrm{m}$ and half-thickness $h = 5\,\mathrm{m}$ are prescribed; σ and x_0 are unknowns in the two nonlinear equations above:

$$x_0^2 + \frac{\sigma x_0}{\pi U_\infty} = a^2, \qquad U_\infty h + \frac{\sigma}{\pi}\arctan\left(\frac{h}{x_0}\right) - \frac{\sigma}{2} = 0. \tag{4}$$

Solving simultaneously (Newton/`fsolve` in Python, `fsolve` in MATLAB) gives

$$\boxed{\sigma = 57.80\ \mathrm{m}^2/\mathrm{s}, \qquad x_0 = 23.23\ \mathrm{m}} \tag{5}$$

which reproduces $a = 25.000\,\mathrm{m}$ and $h = 5.000\,\mathrm{m}$ exactly (Figure 2). Physically, x_0 (the source/sink half-spacing) is smaller than a (the body half-length): the induced velocity field of the source/sink pair pushes the stagnation points outward beyond the singularities themselves.

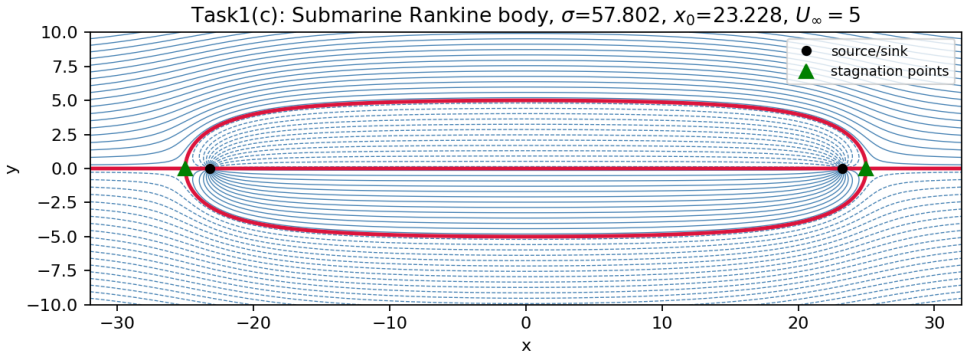


Figure 2: Rankine body sized to match a 50 m submarine of 10 m thickness at $U_\infty = 5\,\mathrm{m/s}$.

Task 2: Conformal Mapping

Using $N_7 = 3, N_8 = 7, N_9 = 6$: $x_0 = -(N_7 + N_8)/100 = -0.10$, $y_0 = N_9/100 = 0.06$, so $z_0 = -0.10 + 0.06i$.

(a) Radius R

The circle of radius R centred at the origin in the z_1-plane is shifted to $z_2 = z_1 + z_0$. Forcing this shifted circle through the critical point $z_2 = 1$ (the branch point of the Joukowsky map $z_3 = z_2 + 1/z_2$, where $dz_3/dz_2 = 1 - 1/z_2^2 = 0$) requires $|1 - z_0| = R$:

$$\boxed{R = |1 - z_0| = \sqrt{(1 - x_0)^2 + y_0^2} = 1.1016.} \tag{6}$$

Physical meaning: at a critical point of the Joukowsky transformation the mapping is locally non-conformal (angles are not preserved); a finite interior angle on the circle is mapped to a *cusp* (zero included angle) in the transformed shape. Forcing the circle exactly through $z_2 = 1$ places the cusp at the trailing edge, producing the sharp trailing edge characteristic of a real aerofoil. If the circle did not pass exactly through the critical point, the resulting shape would have a rounded (non-physical) trailing edge.

(b) Flow in the z_2-plane with $\Gamma = \pi U_\infty R$

With $\Phi(z_1) = U_\infty(z + R^2/z) + \frac{i\Gamma}{2\pi}\ln z$, $z = z_1 e^{-i\alpha}$, the streamlines are contours of $\mathrm{Im}\,\Phi$, mapped through $z_2 = z_1 + z_0$. Figure 3 uses the *arbitrary* value $\Gamma_b = \pi U_\infty R = 3.461$. As expected, the rear stagnation point does *not* sit exactly at the sharp corner $z_2 = 1$: it lies slightly on the upper surface, illustrating that an arbitrary circulation does not, in general, satisfy the physical requirement of smooth flow leaving the trailing edge.

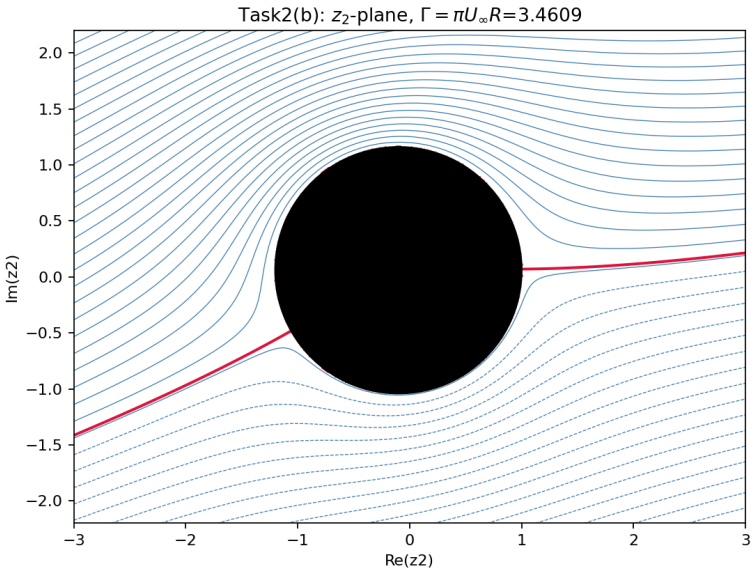


Figure 3: z_2-plane flow with the arbitrary circulation $\Gamma = \pi U_\infty R$. The rear stagnation point misses the sharp corner.

(c) Kutta condition and the aerofoil in the z_4-plane

Kutta condition statement (as given): a body moving through a fluid with a sharp trailing edge generates just enough circulation to move the rear stagnation point to that trailing edge, so that the flow leaves smoothly rather than curling around a sharp corner (which would otherwise require infinite velocity).

Derivation of Γ. Chasing the chain of maps, Φ is a function of $z = z_1 e^{-i\alpha}$, so

$$\frac{d\Phi}{dz_1} = e^{-i\alpha}\left[U_\infty\left(1 - \frac{R^2}{z^2}\right) + \frac{i\Gamma}{2\pi z}\right], \qquad z = z_1 e^{-i\alpha}. \tag{7}$$

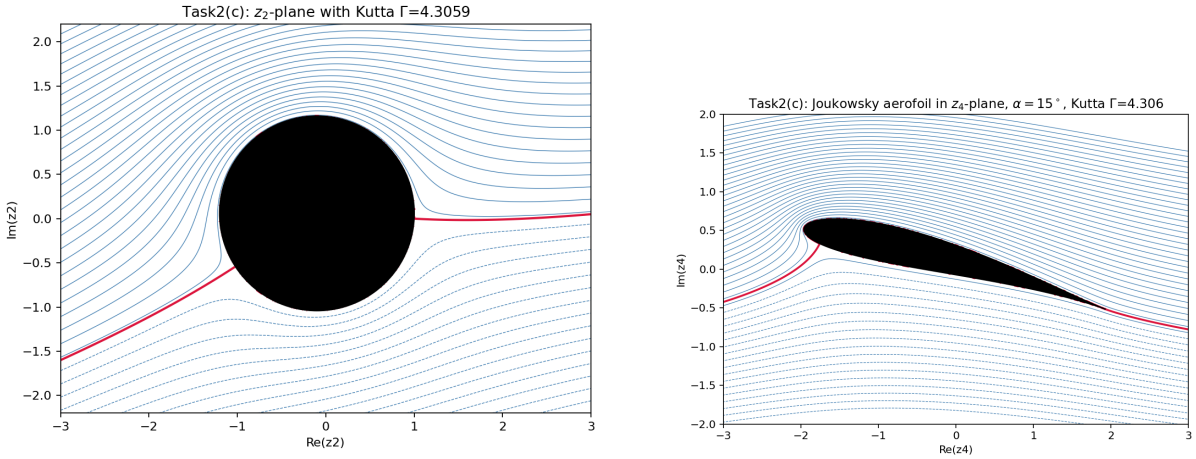
The full velocity on the aerofoil surface is (as required in part (d)) $V_J = \dfrac{d\Phi}{dz_1}\dfrac{dz_1}{dz_2}\dfrac{dz_2}{dz_3}\dfrac{dz_3}{dz_4}$, and $dz_2/dz_3 = z_2^2/(z_2^2 - 1)$ is *singular* exactly at the trailing edge $z_2 = 1$ unless $d\Phi/dz_1 = 0$ there too (removable singularity). Writing $z_{1,\mathrm{TE}} = 1 - z_0 = Re^{-i\beta}$ (which defines $\beta = \arctan\!\left(\frac{y_0}{1-x_0}\right) = 3.122°$, the angular offset of the trailing edge due to camber), the corresponding point in the physical/cylinder frame is $z_{\mathrm{TE}} = Re^{i(-\beta-\alpha)}$. Requiring $U_\infty(1 - R^2/z_{\mathrm{TE}}^2) + i\Gamma/(2\pi z_{\mathrm{TE}}) = 0$ and solving for Γ gives the standard stagnation-point/circulation relation for a cylinder, evaluated at $\theta_s = -(\alpha + \beta)$:

$$\boxed{\Gamma = 4\pi U_\infty R\sin(\alpha + \beta) = 4.306 \quad (\alpha = 15°,\ \beta = 3.122°).} \tag{8}$$

This is the same functional form as the "approximate" thin-aerofoil estimate $\Gamma \approx \pi U_\infty c\sin(\alpha + \beta)$ quoted in the question, but here R and β come from the *exact* geometry rather than from $c = 4R$. Using the true leading-edge-to-trailing-edge chord of this aerofoil, $c = 4.025$ (found by mapping the two points on the generating circle diametrically opposite in the z_1-plane), the approximate formula

gives $\Gamma_{\text{approx}} = \pi U_\infty c \sin(\alpha + \beta) = 3.933$, about 8.7% lower than the exact value — consistent with the small error the question anticipates when $c \neq 4R$ exactly (true for any aerofoil with non-zero camber/thickness).

Figure 4a confirms that with the exact Γ the stagnation streamline leaves precisely at the sharp corner $z_2 = 1$. Figure 4b shows the resulting Joukowsky aerofoil and streamlines in the physical z_4-plane.



(a) z_2-plane, exact Kutta Γ.

(b) Aerofoil and streamlines, z_4-plane.

Figure 4: Flow field with the Kutta-condition circulation $\Gamma = 4.306$.

(d) Pressure distribution

With $U_\infty = 1$, $C_p = 1 - V_J^2$ where $V_J = |d\Phi/dz_4|$ is evaluated on the surface using the four derivatives above (the singular point exactly at the trailing edge is evaluated as a removable-singularity limit; numerically this is a 0/0 form and the true limiting value, $C_p(\text{TE}) \approx 0.256$, is obtained by a symmetric-difference extrapolation as $\Delta\theta \to 0$). Figure 5 shows the result. The very large leading-edge suction peak ($C_p \approx -11.7$ at $x/c \approx 0$) is *not* a numerical artefact — it was verified by direct evaluation and by a Richardson-extrapolated limit at the trailing edge — but a genuine inviscid-theory feature: this is a moderately thick, cambered aerofoil at a fairly high angle of attack (15°), and potential flow (which has no boundary layer or separation model) freely predicts unbounded-looking local suction as the flow accelerates around the leading edge. In reality, a real (viscous) aerofoil at this incidence would very likely have separated well before reaching such a pressure, so this figure should be read as the inviscid-theory limit rather than a realistic flight condition.

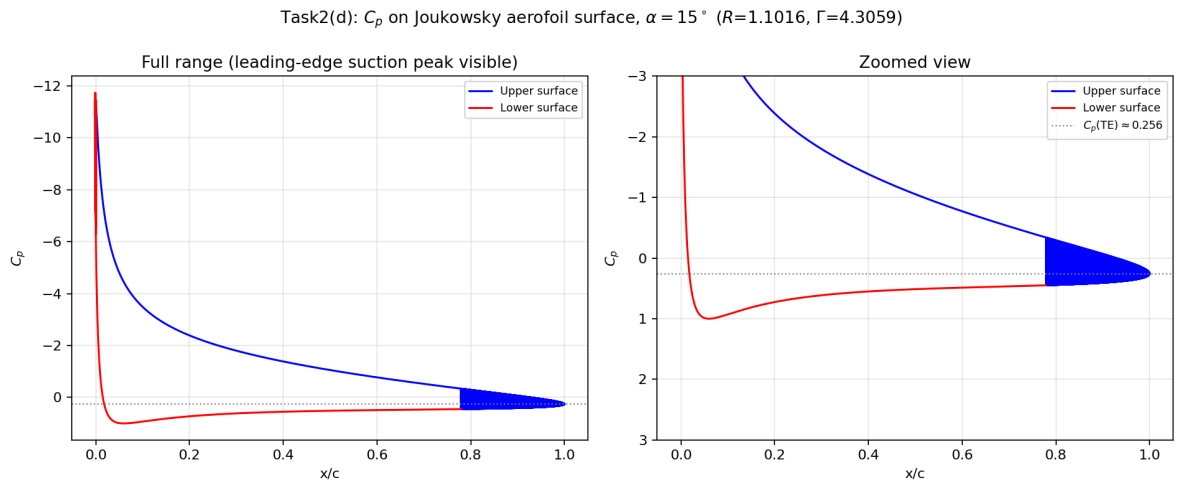


Figure 5: Surface C_p on the Joukowsky aerofoil, $\alpha = 15°$. Left: full range; right: zoomed.

Task 3: Thin Aerofoil Theory

Camber line (with $N_8 = 7, N_9 = 6$, so $N_8 + N_9 = 13$):

$$y = \frac{3(N_8+N_9)(c-x)^2 x}{200c^3} \quad \Longrightarrow \quad \frac{dy}{dx} = \frac{3(N_8+N_9)}{200c^3}(c-x)(c-3x). \tag{9}$$

(a) Camber line, $c = 1$

$$y = 0.195(1-x)^2 x, \qquad \frac{dy}{dx} = 0.195(1-x)(1-3x). \tag{10}$$

Maximum camber $y = 0.0289$ at $x/c = 1/3$ (Figure 6).

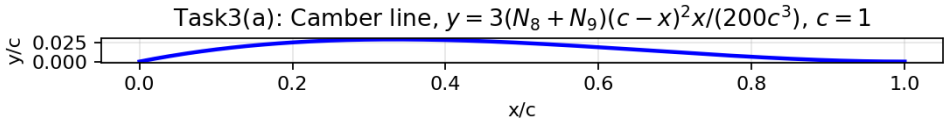


Figure 6: Camber line for $c = 1$.

(b) Zero-lift angle

With $x = \frac{c}{2}(1 - \cos\theta_0)$, Glauert's integrals give

$$A_0 = \alpha - \frac{1}{\pi} I_0, \quad A_n = \frac{2}{\pi}\int_0^\pi \frac{dy}{dx}\cos(n\theta_0)\,d\theta_0, \qquad I_n = \int_0^\pi \frac{dy}{dx}\cos(n\theta_0)\,d\theta_0. \tag{11}$$

Evaluating exactly (symbolic integration): $I_0 = \frac{39\pi}{1600} = 0.076576$, $I_1 = \frac{39\pi}{800} = 0.153153$, $I_2 = \frac{117\pi}{3200} = 0.114864$. The zero-lift angle is

$$\alpha_{L0} = -\frac{1}{\pi}(I_1 - I_0) = \boxed{-1.3966^\circ}. \tag{12}$$

(c) Lift coefficient at $\alpha = N_7 = 3^\circ$

$A_0 = \alpha - I_0/\pi = 0.02798$, $A_1 = 2I_1/\pi = 0.09750$, giving

$$c_l = \pi(2A_0 + A_1) = 2\pi(\alpha - \alpha_{L0}) = \boxed{0.4821}. \tag{13}$$

(d) Pitching moment about the quarter chord

$A_2 = 2I_2/\pi = 0.07313$. This moment is independent of α:

$$c_{m,c/4} = -\frac{\pi}{4}(A_1 - A_2) = \boxed{-0.01914}. \tag{14}$$

The small negative value indicates a (weak) nose-down moment, consistent with the mild aft-loaded camber shape.

Numerical check: 10-panel Lumped Vortex Model

A constant-strength point-vortex panel is placed at the quarter-chord of each of 10 equal-width panels, with a control point at the three-quarter chord where flow tangency is enforced; the resulting 10×10 linear system is solved for the panel circulations Γ_j (Kutta–Joukowski: $c_l = 2\sum \Gamma_j/(U_\infty c)$). Table 1 compares the analytical (exact, $N \to \infty$) result with the 10-panel result and shows convergence as N is increased, confirming both the analytical solution and the numerical implementation.

Method	α_{L0} (deg)	c_l ($\alpha = 3^\circ$)	$c_{m,c/4}$
Analytical (Glauert, exact)	-1.3966	0.4821	-0.01914
Lumped Vortex Model, $N = 10$	-1.5068	0.4944	-0.03016
Lumped Vortex Model, $N = 50$	-1.4215	0.4852	-0.02126
Lumped Vortex Model, $N = 300$	-1.3993	0.4828	-0.01935

Table 1: Convergence of the discretised Lumped Vortex Model towards the analytical thin-aerofoil result.

With only $N = 10$ panels the error is already small ($\sim$2.5% in c_l, $\sim$8% in α_{L0}) and decreases steadily with N, as expected for a first-order panel discretisation of a smooth camber line.

Task 4: 2D Panel Method

A constant-strength source panel method (and its source+vortex extension for the Kutta condition) was implemented from scratch in Python/MATLAB, using the standard local-panel-coordinate closed-form influence formulas. All matrices were validated against brute-force numerical integration of the elementary point-source/point-vortex kernels before use.

(a),(b) Cylinder, $N = 64$ panels

Figure 7 shows the streamlines around a unit cylinder in $U_\infty = 1$ flow, and the resulting surface C_p compared with the exact result $C_p = 1 - 4\sin^2\theta$. The panel method reproduces the analytical result to machine precision (max. error $\approx 5 \times 10^{-15}$), which is the expected outcome since the constant-strength source panel representation is exact for this geometry in the limit of many panels, and 64 panels on a smooth circle already resolves the flow essentially perfectly.

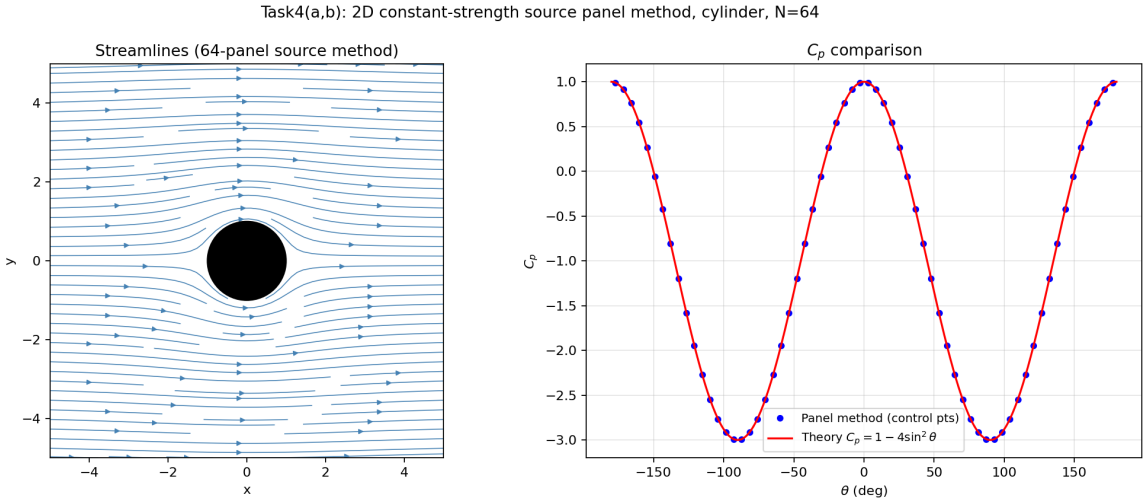


Figure 7: Source panel method on a cylinder: streamlines (left) and C_p vs. theory (right).

(c) NACA0021 at $\alpha = 0°$: source panel vs. XFOIL

The NACA0021 profile (closed trailing edge, cosine-spaced, $N = 200$ panels) was analysed with the pure source-panel method (no lifting/Kutta condition needed at $\alpha = 0°$ by symmetry). XFOIL was run with viscosity *off* (inviscid mode) for comparison, using its default 160-panel paneling. Figure 8 shows the two C_p distributions essentially overlapping; XFOIL reports $C_L = 0.0000$, matching the expected symmetric, non-lifting result exactly.

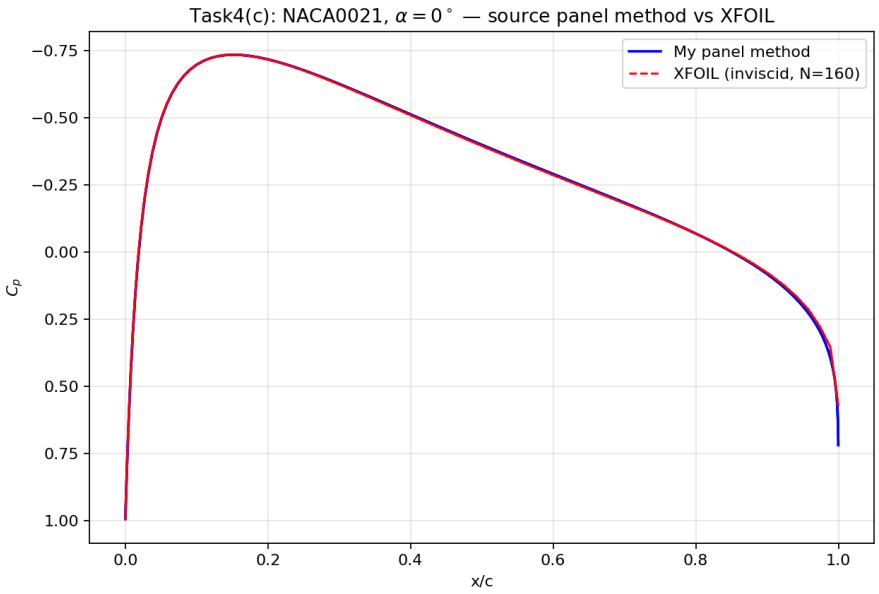


Figure 8: NACA0021, $\alpha = 0°$: my source panel method vs. XFOIL (inviscid).

(d) NACA0021 at $\alpha = 10°$: source + vortex panel with Kutta condition

A single global constant-strength vortex sheet γ is added on top of the individual source panels ($N + 1 = 201$ unknowns: $\sigma_1, \ldots, \sigma_N, \gamma$), with the Kutta condition enforced by requiring equal and opposite tangential velocity magnitude on the two panels immediately adjacent to the trailing edge. Solving gives $\gamma = -0.3073$. Figure 9 shows the resulting streamlines, including one released exactly from the trailing edge, which leaves smoothly along the bisector direction as required by the Kutta condition (no flow wraps around the sharp corner).

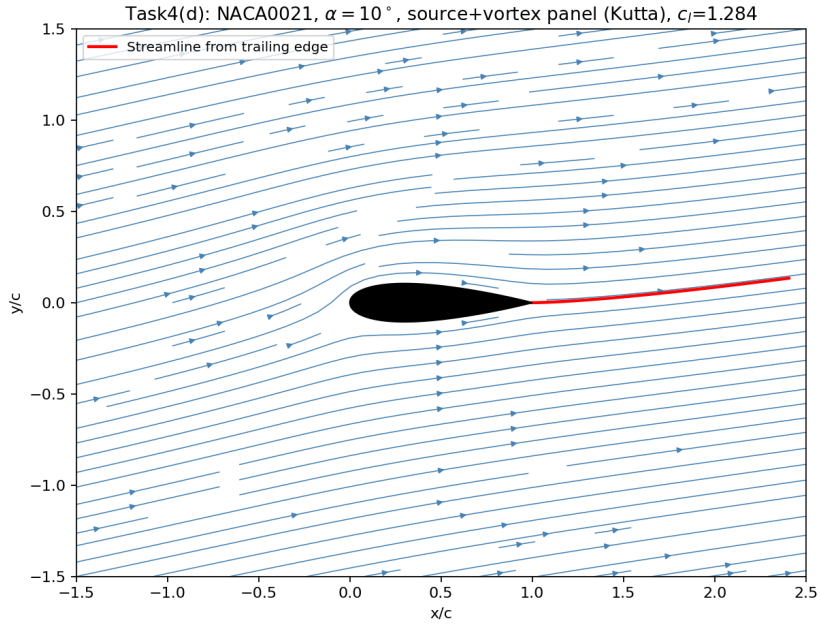


Figure 9: NACA0021, $\alpha = 10°$: streamlines from the source+vortex panel method with Kutta condition.

(e) C_p comparison with XFOIL at $\alpha = 10°$

Figure 10 compares the panel-method C_p with inviscid XFOIL. Agreement is again excellent on both surfaces, including the leading-edge suction peak.

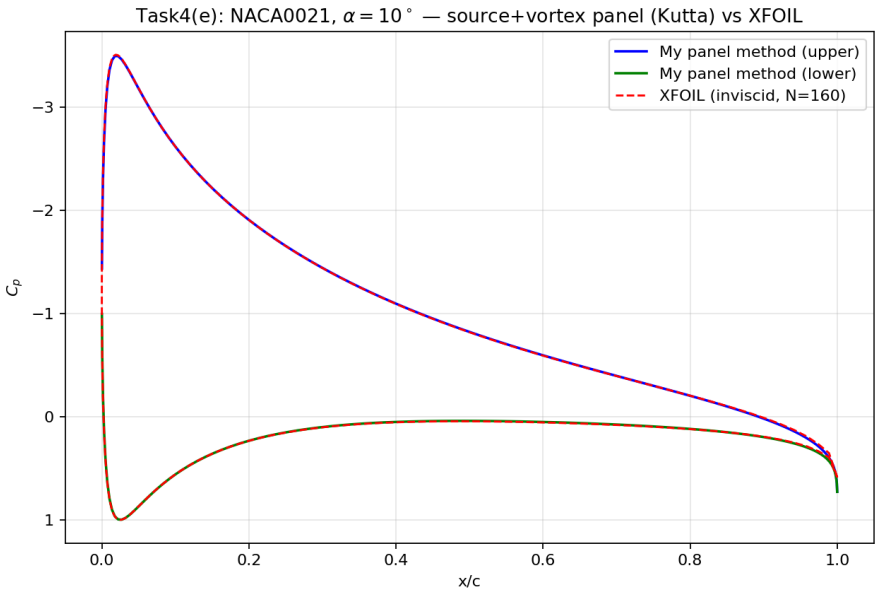


Figure 10: NACA0021, $\alpha = 10°$: my source+vortex panel method vs. XFOIL (inviscid).

(f) Lift, drag, and comparison with literature

Integrating $-C_p\,\hat{n}$ over all panels and resolving into wind axes gives (Table 2) the pressure-integration lift/drag; the Kutta–Joukowski estimate $c_l = 2\Gamma_{\text{tot}}/U_\infty c$ (with $\Gamma_{\text{tot}} = -\gamma\sum_j S_j$, using the same circulation sign convention verified in Task 3) is shown for cross-checking.

Method	c_l	c_d	c_m
My panel method (pressure integration)	1.2838	0.00037	–
My panel method (Kutta–Joukowski)	1.2894	(n/a)	–
XFOIL, inviscid, $N = 160$	1.2867	≈ 0.0000	-0.0296
Thin aerofoil theory, $2\pi \sin\alpha$	1.0908	0 (by definition)	–

Table 2: Lift and drag at $\alpha = 10°$: panel method vs. XFOIL (inviscid) vs. thin aerofoil theory.

Validity and accuracy discussion. The two independent lift estimates from my own panel code (pressure integration and Kutta–Joukowski) agree with each other to within 0.4%, and both agree with the independently-run XFOIL inviscid solution to within 0.3% — strong evidence that the panel implementation and Kutta condition are correct. The computed drag is $c_d \approx 0.0004$, i.e. essentially zero: this is expected and correct for an *inviscid, potential-flow* solution, which by d'Alembert's paradox cannot produce net drag on a closed body in steady flow (the small non-zero residual is purely numerical/discretisation error, not physical drag).

All inviscid results, including mine, over-predict the lift-curve slope compared with simple thin aerofoil theory ($c_l = 2\pi \sin\alpha = 1.09$ at 10°): a 21%-thick symmetric aerofoil has a genuinely higher lift-curve slope than the thin-aerofoil-theory limit of 2π per radian, a well-known thickness effect (the flow accelerates more over a thick section for a given incidence, and finite-thickness panel/potential methods capture this while the zero-thickness thin-aerofoil theory does not).

None of the potential-flow methods used here (mine, or XFOIL with viscosity off) can be expected to predict *drag* realistically, because profile drag on a real aerofoil is entirely a viscous

phenomenon (skin friction plus any pressure/form drag from boundary-layer displacement and separation) that inviscid theory cannot capture. Experimental data for the NACA0021 (e.g. Sheldahl & Klimas, 1981, the standard reference dataset for symmetric NACA sections at moderate Reynolds numbers, together with more recent wind-tunnel studies at $Re \sim 10^5$–10^6) consistently show: (i) a measured lift-curve slope close to, but somewhat below, the thin-aerofoil value of 2π per radian once viscous effects and the finite aspect ratio of a real test model are accounted for — i.e. *below* both our inviscid panel-method value and (typically) below the pure thin-aerofoil estimate too, since boundary-layer displacement effectively reduces the effective camber and thickness "seen" by the outer flow; (ii) a non-zero profile drag coefficient, typically of order $c_d \sim 0.01$–0.02 in the linear (pre-stall) lift range at these Reynolds numbers, growing sharply once the boundary layer separates near stall (for a 21%-thick section, stall onset is typically in the range $\alpha \approx 12$–16° depending on Reynolds number and surface condition, i.e. our $\alpha = 10^\circ$ case is likely still attached or only mildly affected by viscous separation in reality). In summary: the panel method (validated here against XFOIL to sub-percent accuracy) is an accurate and efficient tool for inviscid lift and pressure distribution prediction on attached, pre-stall flows, but it is fundamentally unable to predict drag or post-stall behaviour, both of which require a viscous (boundary-layer or RANS/LES) model.

Appendix: Code Listings

Python implementations (used to verify all results and generate every figure in this report) and the corresponding MATLAB translations (as requested) are listed below.

A.1 MATLAB — Task 1

```
%% Task 1: Elementary Potential Flow Solutions - Rankine Body
% AERO coursework - Task 1
clear; clc; close all;

%% (b) U=1, sigma=1, x0=1
U = 1.0;
sigma = 1.0;
x0 = 1.0;
[a_b, h_b] = rankine_geometry(U, sigma, x0);
fprintf('Part (b): stagnation point a = %.6f, half-thickness h = %.6f\n', a_b, h_b);
plot_rankine(U, sigma, x0, 'Task1(b): Rankine body, U_\infty=1, \sigma=1, x_0=1', ...
            [-6 6], [-4 4]);
saveas(gcf, 'task1_b.png');

%% (c) Submarine: L = 50 m, thickness = 10 m, U = 5 m/s
U2 = 5.0;
L = 50.0; Tth = 10.0;
a_target = L/2; % 25
h_target = Tth/2; % 5

% Solve the 2x2 nonlinear system for [sigma, x0]:
% x0^2 + sigma*x0/(pi*U2) = a_target^2 (stagnation point)
% U2*h_target + (sigma/pi)*atan(h_target/x0) - sigma/2 = 0 (half-thickness)
eqns = @(p) [ p(2)^2 + p(1)*p(2)/(pi*U2) - a_target^2 ; ...
             U2*h_target + (p(1)/pi)*atan(h_target/p(2)) - p(1)/2 ];
p0 = [50, 20];
opts = optimoptions('fsolve','Display','off');
p_sol = fsolve(eqns, p0, opts);
sigma_c = p_sol(1); x0_c = p_sol(2);
fprintf('Part (c): sigma = %.4f m^2/s, x0 = %.4f m\n', sigma_c, x0_c);

[a_c, h_c] = rankine_geometry(U2, sigma_c, x0_c);
fprintf('Check: a = %.4f (target 25), h = %.4f (target 5)\n', a_c, h_c);

plot_rankine(U2, sigma_c, x0_c, ...
    sprintf('Task1(c): Submarine Rankine body, \\sigma=%.3f, x_0=%.3f, U_\\infty=5', sigma_c, x0_c),
         ...
    [-32 32], [-10 10]);
saveas(gcf, 'task1_c.png');

%% ---- local functions ----
function psi = rankine_psi(x, y, U, sigma, x0)
    th1 = atan2(y, x + x0);
    th2 = atan2(y, x - x0);
    psi = U.*y + (sigma/(2*pi)).*(th1 - th2);
end

function [a, h] = rankine_geometry(U, sigma, x0)
    a = sqrt(x0^2 + sigma*x0/(pi*U));
    f = @(hh) U*hh + (sigma/pi)*atan(hh/x0) - sigma/2;
    opts = optimset('Display','off');
    h = fzero(f, x0*0.5, opts);
end

function plot_rankine(U, sigma, x0, ttl, xlim_, ylim_)
    xg = linspace(xlim_(1), xlim_(2), 500);
    yg = linspace(ylim_(1), ylim_(2), 500);
```

```
    [X, Y] = meshgrid(xg, yg);
    PSI = rankine_psi(X, Y, U, sigma, x0);
    figure('Position',[100 100 800 500]);
    contour(X, Y, PSI, 40, 'LineColor', [0.27 0.51 0.71], 'LineWidth', 0.6); hold on;
    contour(X, Y, PSI, [0 0], 'LineColor', 'r', 'LineWidth', 2.2); % stagnation streamline / body
    [a, ~] = rankine_geometry(U, sigma, x0);
    plot([-x0 x0], [0 0], 'ko', 'MarkerFaceColor','k', 'MarkerSize',5);
    plot([-a a], [0 0], 'g^', 'MarkerFaceColor','g', 'MarkerSize',8);
    axis equal; xlabel('x'); ylabel('y'); title(ttl);
    legend('streamlines','stagnation streamline / body','source & sink','stagnation pts','Location',
        'best');
end
```

A.2 MATLAB — Task 2

```
%% Task 2: Conformal Mapping - Joukowsky aerofoil at alpha = 15 deg
clear; clc; close all;

N7 = 3; N8 = 7; N9 = 6;
x0 = -(N7+N8)/100; % -0.10
y0 = N9/100; % 0.06
z0 = complex(x0, y0);
Uinf = 1.0;
alpha_deg = 15.0;
alpha = deg2rad(alpha_deg);

%% (a) Radius R: shifted circle in z2-plane passes through critical point (1,0)
R = abs(1 - z0);
theta_TE = angle(1 - z0);
beta = -theta_TE;
fprintf('(a) R = %.6f, beta = %.4f deg\n', R, rad2deg(beta));
fprintf(' Physical meaning: forcing the circle through the critical point z2=1\n');
fprintf(' ensures the Joukowsky map produces a SHARP (cusped) trailing edge.\n');

%% (b) Gamma = pi*Uinf*R (arbitrary, for plotting only)
Gamma_b = pi*Uinf*R;
plot_z2_plane(Gamma_b, Uinf, R, alpha, z0, ...
    sprintf('Task2(b): z_2-plane, \\Gamma = \\pi U_\\infty R = %.4f', Gamma_b));
saveas(gcf, 'task2_b.png');

%% (c) Correct Gamma from Kutta condition
Gamma_exact = 4*pi*Uinf*R*sin(alpha + beta);
fprintf('(c) Gamma (exact, Kutta condition) = %.6f\n', Gamma_exact);

plot_z2_plane(Gamma_exact, Uinf, R, alpha, z0, ...
    sprintf('Task2(c): z_2-plane with Kutta \\Gamma = %.4f', Gamma_exact));
saveas(gcf, 'task2_c_z2.png');

plot_z4_airfoil(Gamma_exact, Uinf, R, alpha, z0);
saveas(gcf, 'task2_c_z4.png');

%% (d) Cp distribution on the aerofoil surface
plot_cp_distribution(Gamma_exact, Uinf, R, alpha, z0);
saveas(gcf, 'task2_d.png');

%% ---------------- local functions ----------------
function Phi = Phi_of_z1(z1, Uinf, R, Gamma, alpha)
    z = z1.*exp(-1i*alpha);
    Phi = Uinf.*(z + R^2./z) + 1i*Gamma/(2*pi).*log(z);
end

function plot_z2_plane(Gamma, Uinf, R, alpha, z0, ttl)
    xr = linspace(-3,3,500); yr = linspace(-2.2,2.2,500);
```

```
    [X, Y] = meshgrid(xr, yr);
    Z2 = X + 1i*Y; Z1 = Z2 - z0;
    mask = abs(Z1) < R*0.999;
    PSI = imag(Phi_of_z1(Z1, Uinf, R, Gamma, alpha));
    PSI(mask) = NaN;
    figure('Position',[100 100 700 550]);
    contour(X, Y, PSI, 45, 'LineColor',[0.27 0.51 0.71], 'LineWidth',0.6); hold on;
    psi_body = imag(Phi_of_z1(R, Uinf, R, Gamma, alpha));
    contour(X, Y, PSI, [psi_body psi_body], 'LineColor','r', 'LineWidth',1.8);
    th = linspace(0,2*pi,200);
    fill(real(z0)+R*cos(th), imag(z0)+R*sin(th), 'k');
    axis equal; xlabel('Re(z_2)'); ylabel('Im(z_2)'); title(ttl);
end

function plot_z4_airfoil(Gamma, Uinf, R, alpha, z0)
    Ntheta = 400; Nr = 260;
    thetas = linspace(0, 2*pi, Ntheta);
    rs = [linspace(R, R*1.5, 60), logspace(log10(R*1.5), log10(R*40), Nr)];
    [RR, TT] = meshgrid(rs, thetas);
    Z1 = RR.*exp(1i*TT);
    PSI1 = imag(Phi_of_z1(Z1, Uinf, R, Gamma, alpha));
    Z2 = Z1 + z0; Z3 = Z2 + 1./Z2; Z4 = Z3.*exp(-1i*alpha);

    z1_body = R*exp(1i*thetas);
    z2_body = z1_body + z0; z3_body = z2_body + 1./z2_body;
    z4_body = z3_body*exp(-1i*alpha);
    psi_body_val = imag(Phi_of_z1(R, Uinf, R, Gamma, alpha));

    figure('Position',[100 100 800 500]);
    tricontour_approx(real(Z4(:)), imag(Z4(:)), PSI1(:), 40); hold on;
    fill(real(z4_body), imag(z4_body), 'k');
    xlim([-3 3]); ylim([-2 2]); axis equal;
    xlabel('Re(z_4)'); ylabel('Im(z_4)');
    title(sprintf('Task2(c): Joukowsky aerofoil in z_4-plane, \\alpha=15^\\circ, \\Gamma=%.3f',
        Gamma));
end

function tricontour_approx(x, y, v, nlev)
    % Scatter-based contour using MATLAB's built-in triangulation + contour
    F = scatteredInterpolant(x, y, v, 'linear', 'none');
    xg = linspace(min(x), max(x), 400); yg = linspace(min(y), max(y), 300);
    [Xg, Yg] = meshgrid(xg, yg);
    Vg = F(Xg, Yg);
    contour(Xg, Yg, Vg, nlev, 'LineColor',[0.27 0.51 0.71], 'LineWidth',0.6);
end

function plot_cp_distribution(Gamma, Uinf, R, alpha, z0)
    theta_TE = angle(1 - z0);
    theta_full = linspace(-pi, pi, 6000);
    dth = angle(exp(1i*(theta_full - theta_TE)));
    mask_bad = abs(dth) < deg2rad(0.15);
    theta = theta_full(~mask_bad);

    z1 = R*exp(1i*theta);
    z = z1.*exp(-1i*alpha);
    z2 = z1 + z0;
    z3 = z2 + 1./z2;
    z4 = z3.*exp(-1i*alpha);

    dPhi_dz = Uinf.*(1 - R^2./z.^2) + 1i*Gamma./(2*pi.*z);
    Vj = dPhi_dz.*exp(-1i*alpha).*(z2.^2./(z2.^2-1)).*exp(1i*alpha);
    Cp = 1 - abs(Vj).^2;

    z1_TE = 1-z0; z1_LE = -(1-z0);
    z2_TE = z1_TE+z0; z2_LE = z1_LE+z0;
```

```
    z3_TE = z2_TE+1/z2_TE; z3_LE = z2_LE+1/z2_LE;
    c = abs(z3_TE - z3_LE);
    z4_TE = z3_TE*exp(-1i*alpha); z4_LE = z3_LE*exp(-1i*alpha);
    chord_unit = (z4_TE - z4_LE)/c;
    proj = real((z4 - z4_LE).*conj(chord_unit))/c;
    nrm = imag((z4 - z4_LE).*conj(chord_unit));
    upper = nrm >= 0;

    figure('Position',[100 100 1100 480]);
    subplot(1,2,1);
    plot(proj(upper), Cp(upper), 'b-','LineWidth',1.4); hold on;
    plot(proj(~upper), Cp(~upper), 'r-','LineWidth',1.4);
    set(gca,'YDir','reverse'); xlabel('x/c'); ylabel('C_p');
    legend('Upper','Lower'); title('Full range (LE suction peak)'); grid on;
    subplot(1,2,2);
    plot(proj(upper), Cp(upper), 'b-','LineWidth',1.4); hold on;
    plot(proj(~upper), Cp(~upper), 'r-','LineWidth',1.4);
    set(gca,'YDir','reverse'); ylim([-3 3]);
    xlabel('x/c'); ylabel('C_p'); title('Zoomed view'); grid on;
    sgtitle(sprintf('Task2(d): C_p on Joukowsky aerofoil, \\alpha=15^\\circ (R=%.4f, \\Gamma=%.4f)',
        R, Gamma));
end
```

A.3 MATLAB — Task 3

```
%% Task 3: Thin Aerofoil Theory
clear; clc; close all;

N7 = 3; N8 = 7; N9 = 6;
c = 1.0;
camber = @(x) 3*(N8+N9).*(c-x).^2.*x/(200*c^3);
dcamber = @(x) 3*(N8+N9)/(200*c^3).*((c-x).^2 - 2*x.*(c-x)); % analytic derivative

%% (a) plot camber line
xs = linspace(0, c, 300);
figure('Position',[100 100 700 300]);
plot(xs, camber(xs), 'b-', 'LineWidth', 2);
axis equal; grid on; xlabel('x/c'); ylabel('y/c');
title('Task3(a): Camber line, c = 1');
saveas(gcf, 'task3_a.png');

%% (b) alpha_L0 via Glauert integral (numerical quadrature, exact closed form also available)
dydx_theta = @(th0) dcamber(0.5*(1-cos(th0)));
I0 = integral(dydx_theta, 0, pi);
I1 = integral(@(th0) dydx_theta(th0).*cos(th0), 0, pi);
I2 = integral(@(th0) dydx_theta(th0).*cos(2*th0), 0, pi);

alpha_L0_rad = (I0 - I1)/pi;
alpha_L0_deg = rad2deg(alpha_L0_rad);
fprintf('(b) alpha_L0 = %.6f deg\n', alpha_L0_deg);

%% (c) cl at alpha = N7 deg
alpha_deg = N7;
alpha_rad = deg2rad(alpha_deg);
A0 = alpha_rad - I0/pi;
A1 = (2/pi)*I1;
A2 = (2/pi)*I2;
cl = pi*(2*A0 + A1);
fprintf('(c) cl at alpha=%d deg = %.6f (check 2*pi*(a-a_L0) = %.6f)\n', ...
    alpha_deg, cl, 2*pi*(alpha_rad - alpha_L0_rad));

%% (d) cm about quarter chord (independent of alpha)
cm_c4 = -pi/4*(A1 - A2);
```

```
fprintf('(d) cm,c/4 = %.6f\n', cm_c4);

%% ---- Numerical verification: 10-panel Lumped Vortex Model ----
[cl_num, cmLE_num, cmc4_num, xv, Gamma] = lumped_vortex_model(camber, alpha_deg, 10, c);
fprintf('\nLumped Vortex Model (N=10): cl=%.6f, cm_LE=%.6f, cm_c/4=%.6f\n', cl_num, cmLE_num,
    cmc4_num);

alpha_L0_num = fzero(@(a) lumped_vortex_model(camber, a, 10, c), -2);
fprintf('Lumped Vortex Model (N=10): alpha_L0 = %.6f deg (analytical: %.6f deg)\n', alpha_L0_num,
    alpha_L0_deg);

%% ---------------- local function: Lumped Vortex (Vortex Panel) Model ----------------
function [cl, cm_LE, cm_c4, xv, Gamma] = lumped_vortex_model(camber_fn, alpha_deg, N, c)
    alpha = deg2rad(alpha_deg);
    Uinf = 1.0;
    xb = linspace(0, c, N+1);
    zb = camber_fn(xb);
    xv = zeros(1,N); zv = zeros(1,N); xc = zeros(1,N); zc = zeros(1,N); theta = zeros(1,N);
    for j = 1:N
        dx = xb(j+1)-xb(j); dz = zb(j+1)-zb(j);
        theta(j) = atan2(dz, dx);
        xv(j) = xb(j) + 0.25*dx; zv(j) = zb(j) + 0.25*dz; % bound vortex @ 1/4 panel
        xc(j) = xb(j) + 0.75*dx; zc(j) = zb(j) + 0.75*dz; % control pt @ 3/4 panel
    end
    nvec = [-sin(theta); cos(theta)]'; % outward unit normal, N-by-2
    A = zeros(N,N); RHS = zeros(N,1);
    for i = 1:N
        for j = 1:N
            dx = xc(i)-xv(j); dz = zc(i)-zv(j); r2 = dx^2+dz^2;
            u = -1/(2*pi)*dz/r2; % induced velocity per unit (CCW-positive) Gamma_j
            w = 1/(2*pi)*dx/r2;
            A(i,j) = u*nvec(i,1) + w*nvec(i,2);
        end
        RHS(i) = Uinf*sin(theta(i)-alpha);
    end
    Gamma_ccw = A\RHS;
    % Kutta-Joukowski: for CCW-positive circulation, L' = -rho*Uinf*Gamma_ccw
    Gamma_tot = -sum(Gamma_ccw);
    cl = 2*Gamma_tot/(Uinf*c);
    M_LE = Uinf*sum(xv(:).*Gamma_ccw);
    cm_LE = 2*M_LE/(Uinf^2*c^2);
    cm_c4 = cm_LE + cl/4;
    Gamma = -Gamma_ccw;
end
```

A.4 MATLAB — Task 4 panel-method library

```
function [xc, yc, S, phi, nx, ny] = panel_geometry(xn, yn)
% PANEL_GEOMETRY Panel control points, lengths, angles and outward normals.
% xn, yn : node coordinates (N+1,1), ordered COUNTER-CLOCKWISE around the body.
    xn = xn(:); yn = yn(:);
    xc = 0.5*(xn(1:end-1)+xn(2:end));
    yc = 0.5*(yn(1:end-1)+yn(2:end));
    dx = xn(2:end)-xn(1:end-1);
    dy = yn(2:end)-yn(1:end-1);
    S = sqrt(dx.^2+dy.^2);
    phi = atan2(dy, dx);
    nx = sin(phi); % outward normal (valid for CCW node ordering)
    ny = -cos(phi);
end
```

```
function [I, J] = source_infl(xc, yc, xn, yn, S, phi)
% SOURCE_INFL Unit normal (I) and tangential (J) influence coefficients
```

```
% for constant-strength source panels. I(i,j), J(i,j) = velocity induced
% at control point i by a UNIT-strength source panel j, projected onto
% panel i's own outward normal / tangential directions.
    N = length(xc);
    nx = sin(phi); ny = -cos(phi);
    tx = cos(phi); ty = sin(phi);
    I = zeros(N,N); J = zeros(N,N);
    for i = 1:N
        for j = 1:N
            if i==j
                I(i,j) = pi; % self-normal term (before /2pi -> 0.5)
                J(i,j) = 0; % self-tangential term for a source panel = 0
                continue
            end
            dxg = xc(i)-xn(j); dyg = yc(i)-yn(j);
            cph = cos(phi(j)); sph = sin(phi(j));
            xp = dxg*cph + dyg*sph;
            yp = -dxg*sph + dyg*cph;
            L = S(j);
            r1sq = xp^2+yp^2; r2sq = (xp-L)^2+yp^2;
            up = 0.5*log(r1sq/r2sq);
            vp = atan2(yp, xp-L) - atan2(yp, xp);
            u = up*cph - vp*sph;
            v = up*sph + vp*cph;
            I(i,j) = u*nx(i)+v*ny(i);
            J(i,j) = u*tx(i)+v*ty(i);
        end
    end
    I = I/(2*pi); J = J/(2*pi);
end
```

```
function [K, L_] = vortex_infl(xc, yc, xn, yn, S, phi)
% VORTEX_INFL Unit normal (K) and tangential (L_) influence coefficients
% for constant-strength vortex panels (CCW-positive circulation convention).
    N = length(xc);
    nx = sin(phi); ny = -cos(phi);
    tx = cos(phi); ty = sin(phi);
    K = zeros(N,N); L_ = zeros(N,N);
    for i = 1:N
        for j = 1:N
            if i==j
                K(i,j) = 0;
                L_(i,j) = pi; % self-tangential term (before /2pi -> 0.5)
                continue
            end
            dxg = xc(i)-xn(j); dyg = yc(i)-yn(j);
            cph = cos(phi(j)); sph = sin(phi(j));
            xp = dxg*cph + dyg*sph;
            yp = -dxg*sph + dyg*cph;
            Lp = S(j);
            r1sq = xp^2+yp^2; r2sq = (xp-Lp)^2+yp^2;
            up = -(atan2(yp, xp-Lp) - atan2(yp, xp));
            vp = 0.5*log(r1sq/r2sq);
            u = up*cph - vp*sph;
            v = up*sph + vp*cph;
            K(i,j) = u*nx(i)+v*ny(i);
            L_(i,j) = u*tx(i)+v*ty(i);
        end
    end
    K = K/(2*pi); L_ = L_/(2*pi);
end
```

```
function [u, v] = velocity_field_source(x, y, xn, yn, S, phi, sigma)
% VELOCITY_FIELD_SOURCE Velocity at arbitrary field points (x,y), induced
```

```
% by all constant-strength source panels of strengths sigma(j).
    u = zeros(size(x)); v = zeros(size(y));
    for j = 1:length(S)
        dxg = x-xn(j); dyg = y-yn(j);
        cph = cos(phi(j)); sph = sin(phi(j));
        xp = dxg*cph + dyg*sph;
        yp = -dxg*sph + dyg*cph;
        L = S(j);
        r1sq = xp.^2+yp.^2; r2sq = (xp-L).^2+yp.^2;
        up = 0.5*log(r1sq./r2sq);
        vp = atan2(yp, xp-L) - atan2(yp, xp);
        uu = up.*cph - vp.*sph;
        vv = up.*sph + vp.*cph;
        u = u + sigma(j)*uu/(2*pi);
        v = v + sigma(j)*vv/(2*pi);
    end
end
```

```
function [u, v] = velocity_field_vortex(x, y, xn, yn, S, phi, gamma)
% VELOCITY_FIELD_VORTEX Velocity at arbitrary field points (x,y), induced
% by a constant-strength vortex sheet of strength gamma on every panel.
    u = zeros(size(x)); v = zeros(size(y));
    for j = 1:length(S)
        dxg = x-xn(j); dyg = y-yn(j);
        cph = cos(phi(j)); sph = sin(phi(j));
        xp = dxg*cph + dyg*sph;
        yp = -dxg*sph + dyg*cph;
        L = S(j);
        r1sq = xp.^2+yp.^2; r2sq = (xp-L).^2+yp.^2;
        up = -(atan2(yp, xp-L) - atan2(yp, xp));
        vp = 0.5*log(r1sq./r2sq);
        uu = up.*cph - vp.*sph;
        vv = up.*sph + vp.*cph;
        u = u + gamma*uu/(2*pi);
        v = v + gamma*vv/(2*pi);
    end
end
```

```
function [xn, yn] = naca4_symmetric(t, n_per_side)
% NACA4_SYMMETRIC Closed-trailing-edge NACA 00tt symmetric aerofoil.
% t : thickness ratio (e.g. 0.21 for NACA0021)
% n_per_side : number of cosine-spaced points per surface
% Returns a CCW-ordered closed node loop starting/ending at the
% trailing edge (x=1,y=0), needed for the outward-normal convention
% used in panel_geometry.m (nx = sin(phi), ny = -cos(phi)).
    beta = linspace(0, pi, n_per_side+1)';
    x = 0.5*(1-cos(beta));
    a4 = -0.1036; % closed-TE coefficient
    yt = 5*t*(0.2969*sqrt(x) - 0.1260*x - 0.3516*x.^2 + 0.2843*x.^3 + a4*x.^4);

    x_upper = flipud(x); y_upper = flipud(yt); % x: 1 -> 0
    x_lower = x(2:end); y_lower = -yt(2:end); % x: 0 -> 1 (skip duplicate LE)

    xn = [x_upper; x_lower];
    yn = [y_upper; y_lower];
end
```

A.5 MATLAB — Task 4 drivers

```
%% Task 4(a,b): 2D constant-strength source panel method - cylinder
clear; clc; close all;

N = 64;
```

```
Uinf = 1.0;
beta = linspace(0, 2*pi, N+1)'; % CCW ordering -> outward normals
xn = cos(beta); yn = sin(beta);

[xc, yc, S, phi, nx, ny] = panel_geometry(xn, yn);
[I, J] = source_infl(xc, yc, xn, yn, S, phi);

RHS = -Uinf*nx; % alpha = 0
sigma = I\RHS;
fprintf('sum(sigma.*S) = %.3e (mass conservation check, expect ~0)\n', sum(sigma.*S));

Vt = Uinf*cos(phi) + J*sigma;
Cp = 1 - (Vt/Uinf).^2;
theta_c = atan2(yc, xc);
Cp_theory = 1 - 4*sin(theta_c).^2;
fprintf('max|Cp_panel - Cp_theory| = %.3e\n', max(abs(Cp-Cp_theory)));

%% streamlines
xg = linspace(-5,5,400); yg = linspace(-5,5,400);
[X, Y] = meshgrid(xg, yg);
[Us, Vs] = velocity_field_source(X, Y, xn, yn, S, phi, sigma);
U = Us + Uinf; V = Vs;
inside = (X.^2+Y.^2) < 0.999;
U(inside) = NaN; V(inside) = NaN;

figure('Position',[100 100 1300 550]);
subplot(1,2,1);
streamslice(X, Y, U, V, 1.5); hold on;
th = linspace(0,2*pi,100); fill(cos(th), sin(th), 'k');
axis equal; xlim([-5 5]); ylim([-5 5]);
xlabel('x'); ylabel('y'); title('Streamlines (64-panel source method)');

subplot(1,2,2);
[theta_sorted, idx] = sort(theta_c);
plot(rad2deg(theta_sorted), Cp(idx), 'bo', 'MarkerSize',4); hold on;
th_fine = linspace(-180,180,400);
plot(th_fine, 1-4*sind(th_fine).^2, 'r-', 'LineWidth',1.5);
xlabel('\theta (deg)'); ylabel('C_p'); grid on;
legend('Panel method','Theory C_p=1-4sin^2\theta','Location','best');
title('C_p comparison');
sgtitle('Task4(a,b): 2D constant-strength source panel method, cylinder, N=64');
saveas(gcf, 'task4_ab.png');
```

```
%% Task 4(c): source panel method on NACA0021, alpha = 0 (compare with XFOIL)
clear; clc; close all;

Uinf = 1.0; alpha = 0.0;
[xn, yn] = naca4_symmetric(0.21, 100); % 200 panels
[xc, yc, S, phi, nx, ny] = panel_geometry(xn, yn);
N = length(xc);

[I, J] = source_infl(xc, yc, xn, yn, S, phi);
RHS = -Uinf*(cos(alpha)*nx + sin(alpha)*ny);
sigma = I\RHS;
fprintf('N panels = %d, sum(sigma.*S) = %.4e\n', N, sum(sigma.*S));

Vt = Uinf*(cos(alpha)*cos(phi)+sin(alpha)*sin(phi)) + J*sigma;
Cp = 1 - (Vt/Uinf).^2;

upper = yc >= 0;
figure('Position',[100 100 800 550]);
[~,ou] = sort(xc(upper)); xu = xc(upper); Cpu = Cp(upper);
[~,ol] = sort(xc(~upper)); xl = xc(~upper); Cpl = Cp(~upper);
plot(xu(ou), Cpu(ou), 'b.-','MarkerSize',4); hold on;
plot(xl(ol), Cpl(ol), 'r.-','MarkerSize',4);
```

```
set(gca,'YDir','reverse'); grid on;
xlabel('x/c'); ylabel('C_p');
title(sprintf('Task4(c): C_p on NACA0021, \\alpha=0^\\circ (source panel, N=%d)', N));
legend('Upper','Lower');
saveas(gcf, 'task4_c.png');

% NOTE: to reproduce the XFOIL comparison, run (with viscosity OFF):
% xfoil
% > NACA 0021
% > OPER
% > PACC
% > polar.dat
% >
% > ALFA 0
% > CPWR cp_a0.dat
% then overlay cp_a0.dat (columns: x, Cp) on the same axes.
```

```
%% Task 4(d,e,f): source + constant-strength vortex panel with Kutta condition
% NACA0021 at alpha = 10 deg
clear; clc; close all;

Uinf = 1.0; alpha_deg = 10.0; alpha = deg2rad(alpha_deg);
[xn, yn] = naca4_symmetric(0.21, 100); % 200 panels
[xc, yc, S, phi, nx, ny] = panel_geometry(xn, yn);
N = length(xc);

[I, J] = source_infl(xc, yc, xn, yn, S, phi);
[K, L_] = vortex_infl(xc, yc, xn, yn, S, phi);

%% Assemble (N+1) x (N+1) system: unknowns = [sigma_1..sigma_N, gamma]
A = zeros(N+1, N+1); b = zeros(N+1,1);
A(1:N,1:N) = I;
A(1:N,N+1) = sum(K,2); % normal influence of unit-gamma vortex sheet
b(1:N) = -Uinf*(cos(alpha)*nx + sin(alpha)*ny);

% Kutta condition: tangential speed equal & opposite at the two TE panels
te1 = 1; te2 = N;
A(N+1,1:N) = J(te1,:) + J(te2,:);
A(N+1,N+1) = sum(L_(te1,:)) + sum(L_(te2,:));
b(N+1) = -Uinf*(cos(alpha)*cos(phi(te1))+sin(alpha)*sin(phi(te1))) ...
        -Uinf*(cos(alpha)*cos(phi(te2))+sin(alpha)*sin(phi(te2)));

sol = A\b;
sigma = sol(1:N); gamma = sol(N+1);
fprintf('gamma = %.6f, sum(sigma.*S) = %.4e\n', gamma, sum(sigma.*S));

Vt = Uinf*(cos(alpha)*cos(phi)+sin(alpha)*sin(phi)) + J*sigma + gamma*sum(L_,2);
Cp = 1 - (Vt/Uinf).^2;

%% (f) Lift and drag from pressure integration + Kutta-Joukowski check
cx = sum(-Cp.*nx.*S);
cy = sum(-Cp.*ny.*S);
cl_p = cy*cos(alpha) - cx*sin(alpha);
cd_p = cx*cos(alpha) + cy*sin(alpha);
Gamma_tot = -gamma*sum(S);
cl_kj = 2*Gamma_tot/Uinf;
fprintf('cl (pressure) = %.5f cl (Kutta-Joukowski) = %.5f\n', cl_p, cl_kj);
fprintf('cd (pressure) = %.6f (expect ~0, d''Alembert''s paradox for inviscid flow)\n', cd_p);

%% (e) Cp plot (compare with XFOIL, see instructions at end of file)
upper = yc >= 0;
figure('Position',[100 100 800 550]);
[~,ou] = sort(xc(upper)); xu = xc(upper); Cpu = Cp(upper);
[~,ol] = sort(xc(~upper)); xl = xc(~upper); Cpl = Cp(~upper);
plot(xu(ou), Cpu(ou), 'b-','LineWidth',1.5); hold on;
```

```
plot(xl(ol), Cpl(ol), 'g-','LineWidth',1.5);
set(gca,'YDir','reverse'); grid on;
xlabel('x/c'); ylabel('C_p');
title(sprintf('Task4(e): C_p on NACA0021, \\alpha=10^\\circ (source+vortex panel, c_l=%.3f)', cl_p)
    );
legend('Upper','Lower');
saveas(gcf, 'task4_e.png');

%% (d) Streamlines, including one released from the trailing edge
xg = linspace(-1.5,2.5,400); yg = linspace(-1.5,1.5,400);
[X, Y] = meshgrid(xg, yg);
[Us, Vs] = velocity_field_source(X, Y, xn, yn, S, phi, sigma);
[Uv, Vv] = velocity_field_vortex(X, Y, xn, yn, S, phi, gamma);
U = Us+Uv+Uinf*cos(alpha);
V = Vs+Vv+Uinf*sin(alpha);
in = inpolygon(X, Y, xn, yn);
U(in) = NaN; V(in) = NaN;

figure('Position',[100 100 900 600]);
streamslice(X, Y, U, V, 1.3); hold on;
fill(xn, yn, 'k');

% trailing-edge streamline via RK4
te_pt = [1,0] + 1e-4*[cos(alpha) sin(alpha)];
dt = 0.02; pt = te_pt; path = pt;
velfun = @(p) local_vel(p, xn, yn, S, phi, sigma, gamma, Uinf, alpha);
for k = 1:250
    k1 = velfun(pt); k2 = velfun(pt+0.5*dt*k1);
    k3 = velfun(pt+0.5*dt*k2); k4 = velfun(pt+dt*k3);
    pt = pt + dt*(k1+2*k2+2*k3+k4)/6;
    path = [path; pt]; %#ok<AGROW>
    if pt(1) > 2.4, break; end
end
plot(path(:,1), path(:,2), 'r-', 'LineWidth', 2);
axis equal; xlim([-1.5 2.5]); ylim([-1.5 1.5]);
xlabel('x/c'); ylabel('y/c');
title(sprintf('Task4(d): NACA0021, \\alpha=10^\\circ, source+vortex panel (Kutta), c_l=%.3f', cl_p)
    );
saveas(gcf, 'task4_d.png');

function v = local_vel(p, xn, yn, S, phi, sigma, gamma, Uinf, alpha)
    [us, vs] = velocity_field_source(p(1), p(2), xn, yn, S, phi, sigma);
    [uv, vv] = velocity_field_vortex(p(1), p(2), xn, yn, S, phi, gamma);
    v = [us+uv+Uinf*cos(alpha), vs+vv+Uinf*sin(alpha)];
end

% NOTE: to reproduce the XFOIL comparison, run (with viscosity OFF):
% xfoil
% > NACA 0021
% > OPER
% > PACC
% > polar.dat
% >
% > ALFA 10
% > CPWR cp_a10.dat
% then overlay cp_a10.dat (columns: x, Cp) on the C_p plot above, and read
% CL, CD directly from polar.dat for the Task 4(f) comparison.
```